

Programátorská dokumentace – Toads and Frogs

1. Architektura projektu

Projekt je rozdělen do dvou hlavních částí:

- **GameState** – logika hry, práce se stavem, generování tahů, vyhodnocování
- **GUI (Tkinter)** – uživatelské rozhraní, vykreslování, ovládání hry, komunikace s AI

AI pracuje výhradně se stavem hry a nikdy nemění skutečný board, dělá kopii board a tu následně upravuje. GUI pouze rozhoduje, kdy se tah provede.

2. Zadání

Hra Toads and Frogs probíhá na jednorozměrném herním poli, které je reprezentováno jako posloupnost znaků. Každé pole může obsahovat jednu z následujících hodnot:

- **'T'** – toads
- **'F'** – frogs
- **'.'** – prázdné pole

Hráči se střídají v tazích.

Toads (**T**) se mohou pohybovat **doprava** o jedno pole, pokud je prázdné, nebo mohou **přeskočit frog**, pokud je za ní prázdné pole.

Frogs (**F**) se mohou pohybovat **doleva** o jedno pole, nebo mohou **přeskočit toad** stejným způsobem.

Tah je definován jako dvojice pozic (x, y) , kde x je původní pozice figurky a y je cílová pozice.

Hra končí, pokud aktuální hráč nemá žádný legální tah.

Vítězem je hráč, který může provést tah; hráč bez možnosti tahu prohrává.

Velikost herního pole je určena uživatelem v setup okně a musí být kladné celé číslo.

3. AI – Minimax s alfa-beta ořezáváním

3.1 alphabeta(state, depth, alpha, beta)

Rekurzivní funkce simulující tahy do hloubky.

Zastavení rekurze

- Pokud `depth == 0` nebo `state.is_terminal()`, vrací `return evaluate(state), None`

MAX / MIN logika

Určuje se automaticky podle `state.current_player`:

- 'F' - frogs - maximalizuje
- 'T' - toads - minimalizuje

Výběr nejlepšího tahu

Na každé úrovni se:

- vygenerují tahy, pomocí rekurze a funkce `evaluate` se simuluje jejich výsledek, vybere se nejlepší hodnota a ta se vrací nahoru, dokud ji nepřepíše jiná `best_value`

3.2 find_best_move(state, depth)

Vrací nejlepší tah pro AI. GUI následně provede:

`state = state.apply_move(best_move)`, tedy aplikuje nový tah na kopii boardu

4. Vyběr algoritmu

Na hru mého typu je minimax ideální a velice zábavná feature. Jelikož je nastavení herního pole a uživateli v setup okně, může být pole velké, proto vede k výraznému časovému zlepšení alfa-beta pruning, který vymaže ty větve, které by vedly k horšímu ohodnocení než větve již zkoumané. Jednou větví myslíme jeden konkrétní průběh hry. Větve ohodnocuje funkce `evaluate`. Kód se nachází v souboru `ai` ve funkci `alphabeta`. `Alphabeta` tedy zachovává správnost minimaxu, pouze zrychluje výpočetní čas.

5. Program

Program je rozdělen do čtyř modulů: `main.py`, `gamestate.py`, `gui.py` a `ai.py`

Každý modul má jasně definovanou odpovědnost a komunikace mezi nimi probíhá výhradně přes objekty třídy `GameState` a `Move`.

Modul **gamestate.py**

Obsahuje logiku hry Toads and Frogs. Třída `GameState` reprezentuje kompletní stav hry, včetně herního pole, aktuálního hráče a metod pro generování tahů, vyhodnocení stavu a aplikaci tahu. Tah je reprezentován instancí třídy `Move`, která obsahuje počáteční a cílový index. Metoda `apply_move()` je jediný způsob, jak změnit stav hry, tedy změnit aktuální board.

Modul **ai.py**

Implementuje algoritmus minimax s alpha-beta pruning. Funkce **`alphabeta()`** rekurzivně simuluje možné průběhy hry a vrací nejlepší hodnotu stavu pomocí `evaluate()`. Funkce **`find_best_move()`** vrací nejlepší tah pro AI. AI vždy pracuje s kopiemi `GameState` (funkce `clone()`), aby neměnila skutečný stav hry a mohli jsme tak vykreslit až celou změněnou board.

Modul **gui.py**

Zajišťuje uživatelské rozhraní pomocí Tkinteru. Obsahuje vykreslování herního pole, zpracování kliknutí uživatele a volání AI. GUI nikdy nemění stav hry přímo — vždy používá metody `GameState`.

Modul **main.py**

Spouští aplikaci, vytváří hlavní okno a na něm aplikaci.

Komunikace mezi moduly je následující: GUI získává tahy z `GameState`, pokud hraje hráč a jeho tah je v poli, které vrací `generate_moves()`, pak `gamestate` pomocí `apply_move()` aplikuje tah, pokud hraje AI, předává stav AI, to vrací nejlepší tah, GUI jej aplikuje pomocí `apply_move()`.

6. Alternativní programová řešení

Třída **Move**

Původně jsem zamýšlel, že indexy z kterých táhnu a kam táhnu označím jednoduše jednou proměnnou. Ovšem poté se daleko hůře pracuje se zbytkem programu, jelikož tah používám ve více modulech, ať už `gui`, tak `gamestate` i `ai`, se třídou `Move` je to všude přístupné, ale kdybych to měl zdefinované jako proměnnou, musel bych v každém modulu definovat a přenášet ty proměnné, funkce by pak samozřejmě fungovaly, ale bylo by to přes ruku, se třídou `Move` je to intuitivní a dobře přístupné.

update_value()

Dalším upgradem v této funkci bylo zjištění, že existuje funkce `setattr()`, která vybranému jménu přiřadí hodnotu, což je super, protože máme 3 proměnné a každé se může měnit `value`, tedy než abych vypisoval 3 podmínky na `value update`, stačí jeden řádek, který to dělá za nás.

apply_move()

Jediná funkce, která mění vzhled boardu, aplikuje tah na board, důležitou částí zde je ale funkce `clone()`, kde děláme úpravu na klonu boardu v aktuálním stavu, pak se nám zobrazí až celá hotová board. Nemusí to být hned zřejmé, ale bez této funkce by se nám rozbila aplikace, jelikož při tahu AI, by se každý simulovaný tah zobrazil na aktuálním boardu. Buď se tedy dostaneme do nekonečné rekurze nebo dostaneme nic neříkající výsledek, jelikož větve by začínaly každá z jiného stavu.